Flash Attention, Slowly

Starting from TinyGPT

SC26 nixnan tutorial

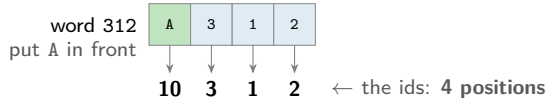
`flash-tutorial`

October 2026

Step 1: a word becomes a list of token ids



A (id 10) = start of a word
F (id 11) = end of a word



the same ids as one-hot rows (4×12)

	0	1	2	3	4	5	6	7	8	9	A	F
A	0	0	0	0	0	0	0	0	0	0	**1**	0
3	0	0	0	**1**	0	0	0	0	0	0	0	0
1	0	**1**	0	0	0	0	0	0	0	0	0	0
2	0	0	**1**	0	0	0	0	0	0	0	0	0

each row has a single 1, in the column of its id

- An id is a **label**: 3 is "token number 3", just as `A` is token number 10. The network never adds or compares ids as numbers.
- The next step turns each id into a vector of 128 numbers by picking one row of a table.

Step 2: each id picks one row of 128 numbers from the embedding table

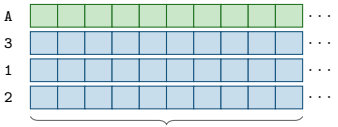
embedding table in `tinygpt.pt` (12×128, after training)

	col 0	col 1	col 2	col 3	⋯	col 127
0	−0.015	+0.010	−0.044	+0.061	⋯	+0.002
1	+0.037	−0.001	−0.059	−0.017	⋯	−0.139
2	+0.013	+0.025	+0.058	−0.007	⋯	+0.004
3	+0.033	+0.016	−0.141	+0.070	⋯	−0.077
4	−0.008	+0.048	+0.008	+0.039	⋯	−0.031
5	+0.030	+0.016	+0.027	+0.009	⋯	−0.013
6	+0.008	+0.066	+0.063	−0.054	⋯	+0.012
7	+0.075	+0.091	+0.013	−0.058	⋯	+0.014
8	+0.035	−0.010	+0.030	−0.076	⋯	+0.022
9	−0.014	−0.014	+0.020	+0.049	⋯	+0.078
A	−0.001	+0.000	−0.006	−0.012	⋯	−0.014
F	+0.010	+0.016	−0.020	+0.020	⋯	+0.005

12 rows: one per token

copy rows → A, 3, 1, 2

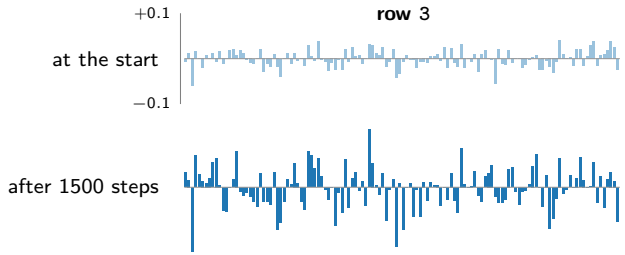
what block 0 gets for `A312`



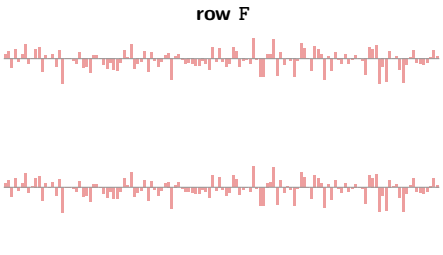
one-hot (4×12) × table (12×128) = these 4 rows (4×128): a matrix multiply that just copies rows

- Every entry is one **FP32 number** (32 bits). A row is 128 numbers, i.e. $128 \times 32 = 4096$ bits, not 128 bits.
- Row 3 is what the network means by 3. Its 128 numbers are coordinates of a point; no single column means anything by itself.

Where the numbers come from: random at first, then moved by training



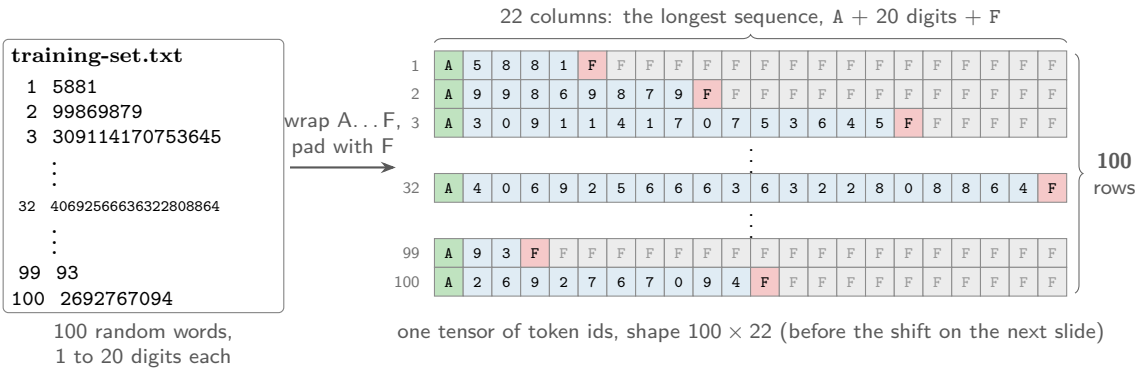
spread out: values now −0.14 to +0.13



unchanged: bit-for-bit the random start

- **Start:** every entry is drawn at random from a normal distribution with mean 0 and spread 0.02 (seed 0).
- **Training:** each of the 1500 AdamW steps nudges every entry in the direction that lowers the loss, exactly like TinyGPT's other parameters (the table is 1,536 of 791,808).
- **Why `F` never moves:** `F` is only ever an *input* after a word has ended, where the target is ignored (−100). Causal attention lets it affect only those positions, so its gradient is exactly zero.

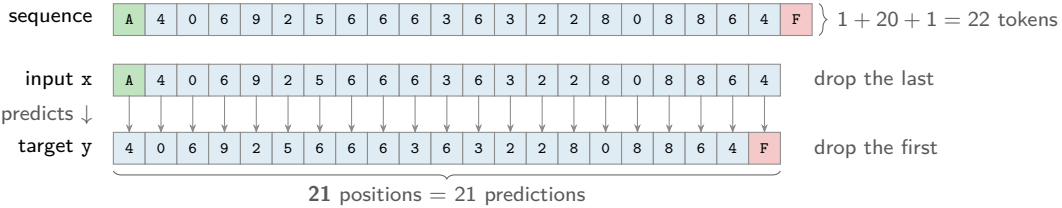
Where the batch size 100 comes from: one row per training word



There is no batch-size setting: `train.py` stacks *every* word of the training set into one batch and takes all 1500 optimizer steps on that same batch (no shuffling, no sampling).

Where 21 comes from: each position predicts the next token

word 32 (20 digits, the longest)

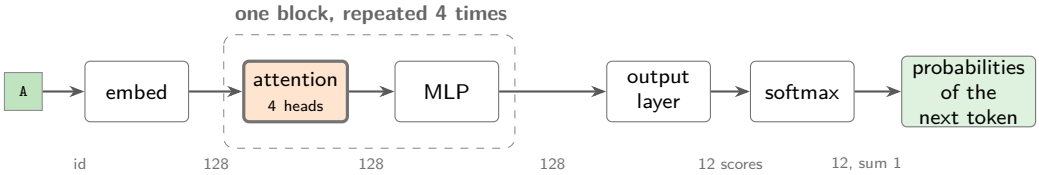


word 1 (5881): padded to the same 21

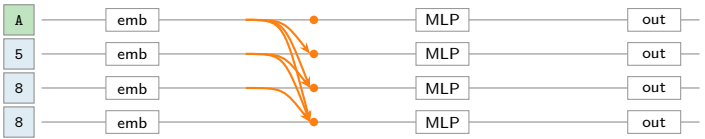


So the training batch is 100×21: 100 words, 21 positions each.

TinyGPT in miniature: what happens to one position



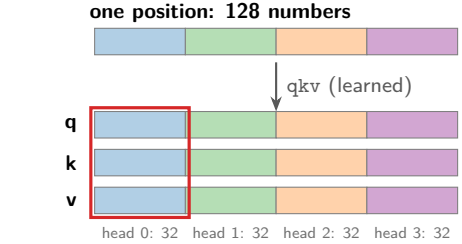
four positions of `A5881F`, side by side:



only the orange attention arrows cross between positions, and only from **earlier** ones

Every box except attention works on each position by itself. Attention is how position 3 learns what came at positions 0, 1, 2.

Inside attention: the 128 numbers are split into 4 heads of 32



head 0 works alone, on its 32-number slices

for every position i (query) and every **earlier or same** position j (key):

$\text{score}_{ij} = q_i \cdot k_j / \sqrt{32}$ (32 products, summed)

weights: softmax of the scores along row i

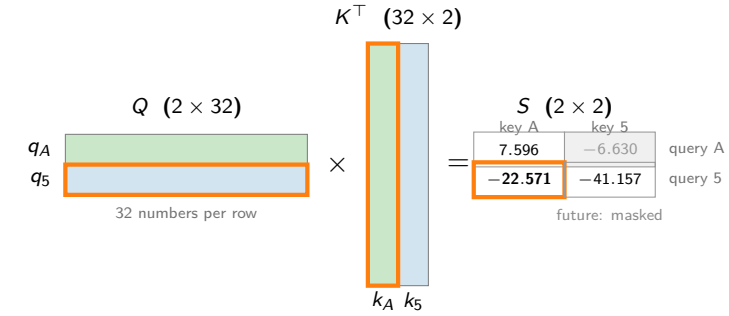
output_i = weighted blend of the v_j (32 numbers)

heads 1, 2, 3: the same, on their own slices, in parallel

glue back to 128, then `proj` mixes the heads → 128 again

A head does *not* make a prediction. It only decides what each position borrows from earlier positions; the prediction comes out of the output layer at the very end. ("Head" and "output head" are unrelated names.)

Attention on A5, step 1: scores $S = Q\,K^\top$ (head 0, block 0)



In training this is the **top-left** 2×2 **corner** of each head's 21×21 grid.

Rows A and 5 may only use keys A and 5 (causal), so the other 19 positions of the word cannot change these numbers.

Each entry: one query row meets one key column.

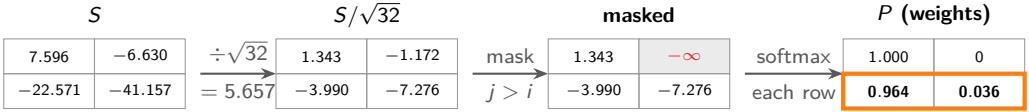
one pair:

	1	2	3	4	⋯	32
q_5	0.673	3.543	5.020	−2.008	⋯	2.209
k_A	6.453	−9.391	−0.717	−4.605	⋯	−0.437
	×	×	×	×		×
products	4.342	−33.271	−3.598	9.247	⋯	−0.965

sum of 32 → **−22.571** $= S_{5A}$

Head 0 of block 0, trained `tinygpt.pt`; q, k after RoPE, all rounded to FP16 as handed to the flash kernel. $q_5 \cdot k_A = -22.6$ is larger than $q_5 \cdot k_5 = -41.2$: query 5 prefers key A.

Step 2: divide by $\sqrt{32}$, mask the future, softmax each row



Why $\sqrt{32}$? A score is a sum of 32 products. If they were independent with spread 1, the sum's spread would be $\sqrt{32}$ (spreads add in quadrature). Dividing by $\sqrt{32}$ keeps scores near ± 1 at the start of training, so the softmax is not already one-hot and gradients can flow.

The 32 is the head size, not the 128 of the whole position.

Row 5 by hand (row A has one key, so its weight is 1):

subtract the row max: $(-3.990, -7.276) - (-3.990) = (0, -3.286)$

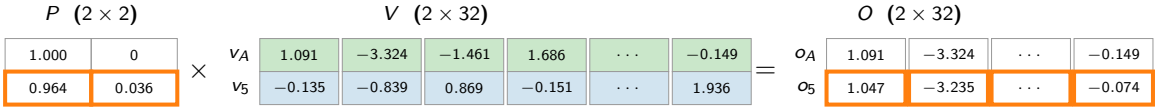
exponentiate: $(1,\ e^{-3.286}) = (1,\ 0.0374)$

divide by the sum 1.0374: $(0.964,\ 0.036)$

These are the 0.96/0.04 of the $n = 2$ snapshot.

Subtracting the row max changes nothing mathematically but keeps $e^x \leq 1$ in FP16/FP32. Flash keeps exactly this max as a *running* max while it streams over key blocks (the online softmax).

Step 3: $O = P\,V$, each output row is a weighted blend of value rows



Row 5, column by column: $o_5 = 0.964\,v_A + 0.036\,v_5$

col 1: $0.964 \times 1.091 + 0.036 \times (-0.135) = 1.047$

col 2: $0.964 \times (-3.324) + 0.036 \times (-0.839) = -3.235$

col 32: $0.964 \times (-0.149) + 0.036 \times 1.936 = -0.074$

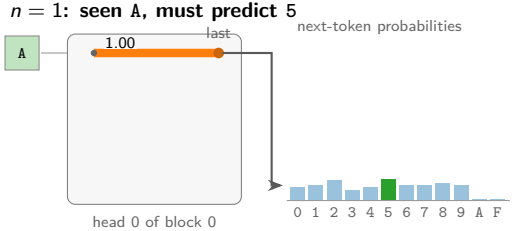
Row A has weight 1 on itself, so $o_A = v_A$ exactly. Position 5 borrows almost everything from A: this head looks back at the start symbol.

Then: heads 1–3 do the same on their own slices; the four 2×32 outputs are glued into 2×128, mixed by `proj`, and added back to x.

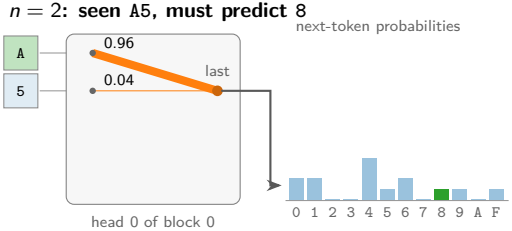
Sizes in training: per word and head, S, P are 21×21 and O is 21×32; per step 100×4 of each, in each of 4 blocks.

Flash attention does steps 1–3 tile by tile on tensor cores (FP16 in, FP32 accumulate): a tile of S lives only in registers, is scaled, masked and exponentiated, multiplied by a tile of V and added into O. S and P are never written to memory.

Training snapshots on A5881F: $n = 1$ and $n = 2$ tokens seen



$p(5) = 0.14$, loss $-\ln 0.14 = 1.97$
14 of the 100 words start with 5: the model has learned how often each first digit occurs

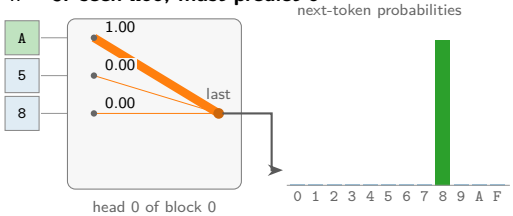


$p(8) = 0.07 = 1/14$, loss 2.64
of the 14 words starting with 5, only one goes on with 8 (and four with 4): A5 alone can't tell

Orange lines: how much the last position's head 0 (block 0) takes from each position seen so far (the numbers sum to 1). A position can never look at a later one. All values are from the trained `tinygpt.pt`.

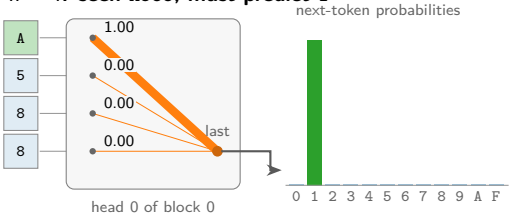
Training snapshots on A5881F: $n = 3$ and $n = 4$ tokens seen

$n = 3$: seen A58, must predict 8



$p(8) = 1.00$, loss 0.00
5881 is the only word starting 58,
so three tokens pin it down

$n = 4$: seen A588, must predict 1



$p(1) = 1.00$, loss 0.00
(and at $n = 5$, A5881 → F, also with $p = 1.00$)

Block 0's head 0 mostly looks back at A; other heads and later blocks pick up the digits. The loss pushes p(correct next token) up at every position, and the gradient flows back through all of it, the embedding rows included.

Training does every snapshot at once

head 0, block 0, word `A5881F`: one row per snapshot

	A	5	8	8	1	F	
A	1.00						→ predict 5
5	0.96	0.04					→ predict 8
8	1.00	0.00	0.00				→ predict 8
8	1.00	0.00	0.00	0.00			→ predict 1
1	0.01	0.88	0.00	0.00	0.11		→ predict F
F	0.40	0.06	0.08	0.03	0.09	0.33	(ignored)

row n = snapshot n:
the last position's weights;
gray = later positions,
never looked at

One forward pass computes all the snapshots together:

- every word: **21** positions = 21 snapshots (one row each)
- every head: one 21×21 grid per word, like the one at left
- per step: 100 words $\times$ 4 heads $\times$ 4 blocks = **1,600 grids**
- loss = average of $-\ln p$(correct next token) over all real positions
- one backward pass, one AdamW step; repeat 1500 times

Generation is different: batch 1, one snapshot at a time. 312 → `A312` (4 positions) in one call, then one new position per call, reusing the stored keys and values of earlier positions.

The educational flash loop (pdf-papers/flash-attention.png)

```
def flash_attention(q, k, v, kv_block_size=2):
    n_q, d = q.shape
    n_k = k.shape[0]
    scale = 1.0 / math.sqrt(d)

    running_max = q.new_full((n_q, 1), -torch.inf)  # m
    running_sum = q.new_zeros(n_q, 1)               # l
    accumulator = q.new_zeros(n_q, v.shape[1])      # U

    for start in range(0, n_k, kv_block_size):
        k_tile = k[start:start + kv_block_size]
        v_tile = v[start:start + kv_block_size]

        scores = q @ k_tile.T * scale                       # (1)
        new_max = torch.maximum(running_max,
                    scores.max(-1, keepdim=True).values) # (2)
        old_scale = torch.exp(running_max - new_max)     # (3)
        tile_weights = torch.exp(scores - new_max)       # (4)
        running_sum = old_scale * running_sum \
                    + tile_weights.sum(-1, keepdim=True) # (5)
        accumulator = old_scale * accumulator \
                    + tile_weights @ v_tile              # (6)
        running_max = new_max

    return accumulator / running_sum                     # (7)
```

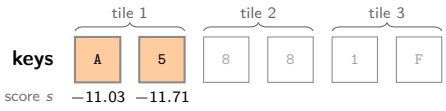
The notepad (one per query row): $m =$ largest score so far, $\ell =$ sum of e^{s-m} so far, $U =$ sum of $e^{s-m}\,v$ so far (32 numbers).

Our run:

- word `A5881F`, query `F` (position 5): it may see all 6 keys, so the code needs no causal mask
- block 2, head 1 of the trained `tinygpt.pt`; $d = 32$, scale $= 1/\sqrt{32} = 0.1768$
- tiles of 2 keys: `[A,5]` `[8,8]` `[1,F]`
- why this head: its running max rises in *every* tile, so steps ③ and ⑥ have work to do. (In block 0, head 0 the largest score is in tile 1, and ③ is always $e^0 = 1$.)

Start: $m = -\infty$, $\ell = 0$, $U = (0, \ldots, 0)$.

Tile 1: keys A and 5



before
m $-\infty$
ℓ 0
U $(0, 0, \ldots, 0)$

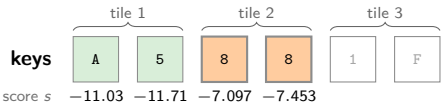
→

after tile 1
m -11.03
ℓ 1.506
U $(-1.043, 0.956, \ldots, 0.416)$

①	`scores`	$q_F \cdot k_A = -62.39,\ q_F \cdot k_5 = -66.24;\ \times 0.1768 \Rightarrow s = (-11.03,\ -11.71)$
②	`new_max`	$\max(-\infty,\ -11.03,\ -11.71) = -11.03$
③	`old_scale`	$e^{-\infty} = 0$ (nothing on the notepad yet)
④	`tile_weights`	$(e^{0},\ e^{-0.68}) = (1,\ 0.5065)$
⑤	`running_sum`	$0 \cdot 0 + (1 + 0.5065) = 1.506$
⑥	`accumulator`	$0 + 1 \cdot v_A + 0.5065 \cdot v_5$; col 1: $1 \cdot (-0.932) + 0.5065 \cdot (-0.221) = -1.043$

The weights are measured against the best score *seen so far* (-11.03), not the true row max, which the loop cannot know yet.

Tile 2: keys 8 and 8, and the first rescale



before
m -11.03
ℓ 1.506
U $(-1.043,\ 0.956,\ \ldots,\ 0.416)$

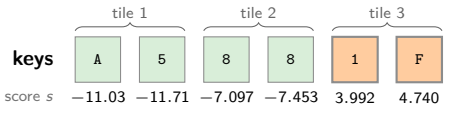
→

after tile 2
m -7.097
ℓ 1.730
U $(-0.959,\ -0.264,\ \ldots,\ 0.834)$

①	`scores`	$(-40.15,\ -42.16) \times 0.1768 \Rightarrow s = (-7.097,\ -7.453)$
②	`new_max`	$\max(-11.03,\ -7.097,\ -7.453) = -7.097$ **the max went up**
③	`old_scale`	$e^{-11.03-(-7.097)} = e^{-3.93} = 0.0196$
④	`tile_weights`	$(e^{0},\ e^{-0.356}) = (1,\ 0.7006)$
⑤	`running_sum`	$0.0196 \cdot 1.506 + (1 + 0.7006) = 0.0295 + 1.7006 = 1.730$
⑥	`accumulator`	$0.0196\,U + v_8 + 0.7006\,v_8'$; col 1: $0.0196 \cdot (-1.043) - 0.640 + 0.7006 \cdot (-0.426) = -0.959$

Tile 1's weights were $e^{s-(-11.03)}$; multiplying by $0.0196 = e^{-11.03-(-7.097)}$ turns them into $e^{s-(-7.097)}$, i.e. measured against the new max. Old work is **rescaled, not redone**: keys `A`, 5 are never read again.

Tile 3: keys 1 and F, the max jumps again



before
m -7.097
ℓ 1.730
U $(-0.959, -0.264, \ldots, 0.834)$

→

after tile 3
m 4.740
ℓ 1.473
U $(-0.618, 0.415, \ldots, -0.022)$

①	`scores`	$(22.58,\ 26.82) \times 0.1768 \Rightarrow s = (3.992,\ 4.740)$
②	`new_max`	$\max(-7.097,\ 3.992,\ 4.740) = 4.740$
③	`old_scale`	$e^{-7.097-4.740} = e^{-11.84} = 0.0000072$
④	`tile_weights`	$(e^{-0.748},\ e^{0}) = (0.473,\ 1)$
⑤	`running_sum`	$0.0000072 \cdot 1.730 + (0.473 + 1) = 1.473$
⑥	`accumulator`	$0.0000072\,U + 0.473\,v_1 + v_F$; col 1: $\approx 0 + 0.473 \cdot 0.327 - 0.773 = -0.618$

The rescale shrinks everything from tiles 1–2 by 7×10^{-6}: those four keys turn out to matter almost not at all. The loop only learns this at tile 3, and fixes the notepad with one multiply.

Step ⑦: divide once, and check against plain attention

⑦ `return accumulator / running_sum`

$U/\ell = (-0.618,\ 0.415,\ -1.124, \ldots, -0.022) \,/\, 1.473$
$= (\mathbf{-0.4195},\ \mathbf{0.2815},\ \mathbf{-0.7632}, \ldots, \mathbf{-0.0146})$

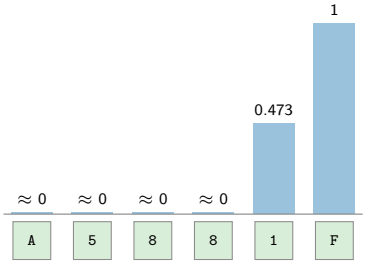
Plain attention (the A5 slides' recipe, all 6 keys at once):

$P = \mathrm{softmax}(s)$
$= (10^{-7},\ 5{\times}10^{-8},\ 5{\times}10^{-6},\ 3{\times}10^{-6},\ 0.321,\ 0.679)$

$PV = (\mathbf{-0.4195},\ \mathbf{0.2815},\ \mathbf{-0.7632}, \ldots, \mathbf{-0.0146})$

Same 32 numbers; in FP64 they differ by 2×10^{-16}. The loop is **exact**, not an approximation.

what the notepad holds at the end



bars: $e^{s_j - m}$ with the final $m = 4.740$;
ℓ = their sum = 1.473, $U = \sum e^{s_j - m} v_j$

Why it works: after every tile, $\ell = \sum e^{s_j - m}$ and $U = \sum e^{s_j - m} v_j$ over the keys read so far, with m the current max. Changing m only changes the reference point, and ③ moves both sums to the new one. In U/ℓ the reference cancels.

From the 28-line loop to PyTorch's flash kernel

The same loop, scaled up:

- **many query rows at once:** one thread block owns a tile of queries, each row with its own m, ℓ, U in registers
- **big key tiles:** for head size 32, PyTorch's kernel uses 128 queries $\times$ 128 keys per tile (`Flash_fwd_kernel_traits<32, 128, 128, 4, ...>` in the nixnan logs)
- ① and ⑥ are matrix multiplies on **tensor cores** (`HMMA`): FP16 in, FP32 out; m, ℓ, U stay FP32
- the **causal mask** is applied to the score tile ($-\infty$ for $j > i$), and tiles entirely in the future are skipped
- S and P never leave the chip; only O (and the lse $= m + \log \ell$, kept for the backward pass) are written

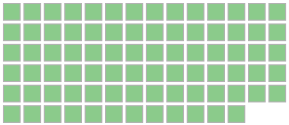
What this means for TinyGPT:

- a word has at most 21 positions, so all its keys fit in **one** 128-key tile
- the kernel runs the loop body **once**: ③ multiplies an empty notepad, and the rescale never matters
- the rescale earns its keep only for sequences longer than one tile (and in split-KV decoding, where partial notepads from several thread blocks are merged with the same $e^{m_{\text{old}} - m_{\text{new}}}$ trick)
- in `logs/smoke.log`, nixnan's reports name these steps' instructions: `HMMA.16816.F32` for ①/⑥, `MUFU.EX2` for ③/④, `FADD`/`FFMA`/`FMUL` for ⑤ and the rescales

Why split the keys? Decoding one token leaves the GPU almost idle

training: 100 words × 4 heads, 21 queries each

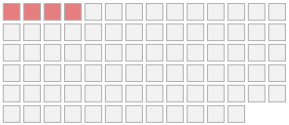
grid $(1, 100, 4) =$ **400 thread blocks**



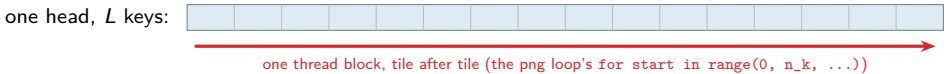
all 82 SMs busy (400 blocks spread over them);
each block: 21 queries × 21 keys

decoding: 4 heads, 1 new query

unsplit: grid $(1, 1, 4) =$ **4 thread blocks**

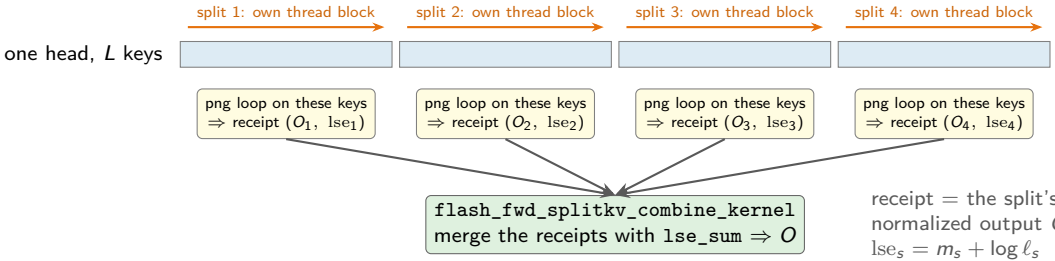


4 of 82 SMs work, 78 wait;
each block walks *all* the keys of its head



Flash attention parallelizes over (query tiles, sequences, heads). Training has plenty of those. A decoding step has one query per head: the work is a long walk over the KV cache, and almost nothing runs in parallel.

Split-KV: several thread blocks share one row's keys



Measured here (PyTorch 2.3.1, RTX 3090, 1 sequence × 4 heads, 1 query, head size 32; times vary by about 1 µs run to run; `split_kv_probe.py`):

keys L	kernels launched	splits	thread blocks	time
7 (TinyGPT 312)	`flash_fwd_kernel`	1	4	
256	`flash_fwd_kernel`	1	4	10 µs
320–512	`flash_fwd_splitkv_kernel` + `..._combine_kernel`	2	8	6–7 + 2 µs
1024	same	4	16	6 + 2 µs
2048	same	8	32	7 + 3 µs
8192	same	16	64	11 + 5 µs
8192, but 100 sequences	`flash_fwd_kernel` (400 blocks already)	1	400	

The combine kernel on our three tiles as three splits

Receipts (query `F`, block 2 head 1; each split runs the png loop on 2 keys):

split	m_s	ℓ_s	lse_s	O_s col 1
1: `A,5`	-11.03	1.506	-10.62	-0.693
2: `8,8`	-7.097	1.701	-6.566	-0.552
3: `1,F`	4.740	1.473	5.128	-0.4195

$\mathrm{lse}_s = m_s + \log \ell_s = \log \sum_{j \in s} e^{s_j}$: the log of the split's **row total** (the kernel's `lse_accum`). Each O_s is a correct answer for its 2 keys only.

Combine kernel (names as in `flash_fwd_kernel.h`):

`lse_max` $= \max_s \mathrm{lse}_s = 5.128$

`lse_sum` $= \sum_s e^{\mathrm{lse}_s - \texttt{lse_max}}$
$= e^{-15.75} + e^{-11.69} + e^{0} = \mathbf{1.0000085}$

`lse_logsum` $= \log(\texttt{lse_sum}) + \texttt{lse_max} = \mathbf{5.128}$

shares $e^{\mathrm{lse}_s - \texttt{lse_logsum}}$
$= (1.4\times10^{-7},\ 8.3\times10^{-6},\ 1.000)$

$O = \sum_s \mathrm{share}_s\, O_s$, col 1 $= -0.4195$:
the single loop's U/ℓ (within 7×10^{-16}, FP64)

`lse_sum` is the row's total $\sum_j e^{s_j}$ measured against `lse_max` (here $\approx$ 1: split 3 dominates); `lse_logsum` is the log of the true total, i.e. the whole row's lse, and is what the kernel stores. A split's share is its fraction of the total; the shares match the softmax weights summed per split.

Where `lse_sum` sits in the png code

png loop (keys tile by tile, one notepad)	**split-KV combine** (whole splits, one receipt each)
line 21 `new_max = max(...)`	`lse_max` $= \max_s \mathrm{lse}_s$
line 22 `old_scale = exp(...)`	the factors $e^{\mathrm{lse}_s - \texttt{lse_max}}$
line 24 `running_sum = ...`	`lse_sum`: row totals added at a common reference, **the same job**
$m + \log($`running_sum`$)$ (not in the code)	`lse_logsum` $= \log($`lse_sum`$) +$ `lse_max`, stored as the row's lse
line 25 `accumulator = ...`	$O = \sum_s e^{\mathrm{lse}_s - \texttt{lse_logsum}} O_s$
line 28 `accumulator / running_sum`	already inside the shares: $-$`lse_logsum` in the exponent is the division

Same idea, different bookkeeping. The loop keeps m and ℓ apart and adds 2 keys at a time. The combine receives each split's total as one number, its log lse_s; `lse_sum` re-adds those totals exactly as line 24 does, and `lse_logsum` turns the result back into a log.

The png code never needs it: it has a single worker that walks every tile.

TinyGPT: generation sees at most 7 keys, so it never splits; the combine (and `lse_sum`) appear from $L > 256$ keys on this GPU.

Next: the one line where the combine checks `lse_sum`.

The guard on `lse_sum` (pdf-papers/lse_sum_gpt_notes.pdf)

```
// all splits -inf  =>  lse_max = 0
lse_max = lse_max == -INFINITY ? 0.0f : lse_max;
// lse_sum = sum_l expf(lse_accum(l) - lse_max)
ElementAccum lse_logsum =
  (lse_sum == 0.f || lse_sum != lse_sum)
    ? INFINITY                  // empty or NaN
    : logf(lse_sum) + lse_max;
gLSE(row) = lse_logsum;         // the stored lse
// each split's share:
sLSE[l][row] = expf(lse_accum(l) - lse_logsum);
```

`flash_fwd_kernel.h` combine kernel, condensed. `x != x` is true only for NaN.

Case 1: `lse_sum == 0`, the row has no visible key (fully masked). Every split reports $\mathrm{lse}_s = -\infty$ (the split kernel's own guard, `softmax.h:177`), `lse_max` becomes 0, and $e^{-\infty} = 0$, so `lse_sum` $= 0$. In png terms: `running_sum` stays 0 and line 28 computes $0/0 =$ NaN.

if `lse_logsum` were $\log 0 = -\infty$ (the "right" value):
share $= e^{-\infty-(-\infty)} = e^{\mathrm{NaN}} =$ **NaN**
$\Rightarrow$ the empty row's output becomes NaN

with the sentinel `lse_logsum` $= +\infty$:
share $= e^{-\infty-(+\infty)} = e^{-\infty} = 0$ for every split
$\Rightarrow$ output row $= 0$, stored lse $= +\infty$, no NaN

Case 2: `lse_sum` is NaN: some lse_s is NaN, or $+\infty$ (then `lse_max` $= +\infty$ and $e^{\infty-\infty} =$ NaN).
The guard keeps the *stored lse* from being NaN ($+\infty$ instead). It does not repair the output: that split's share $e^{\mathrm{NaN}-\infty}$ is still NaN.

With finite inputs and a positive scale every lse_s is finite or $-\infty$; $+\infty$ needs a negative softmax scale (the `nan-flash-attention` deck).